2 Architectural Documentation

2.1 Description of the system architecture

Memory game is a single-player educational cross-platform game developed using the Godot Engine. It is designed for children aged 3 to 7 years and supports touch as well as the mouse. System architecture is based on a modular component-based approach, which makes it simple to scale,share across platforms, and seamlessly integrate visual and sound content.

The structure is partitioned into game logic core, user interface, input abstraction, and asset management, providing a separation of concerns and simplicity of maintenance and testing.

2.1.1 Functional and Non-Functional Requirements :

Functional Requirements :

- System shall allow players to launch games from their device.
- App shall display main menu upon launch
- Select difficulty level (8 to 24 card pairs)
- System must display a GameBoard with cards and Hud based on the selected level.
- System shall allow only two cards to be flipped at a time
- System checks if both flipped cards match.
- The system eliminates card pairs matched.
- system must flip back the non matched card pairs after a short delay.
- System shall calculate the score based on clicks and time and display it at the end.
- the game should end when all cards are matched.
- Responsive UI across devices
- Play sound effects responsively to the input and allow for a mutable theme Background music.
- System shall take touch input for all operations in Game.(only in touch screen supported devices).

Non-Functional Requirements

- Simple, easy-to-use UI for young children
- Platform compatibility: Works on both desktop (mouse) and mobile devices (touch)
- Smooth animations and visual consistency
- Safe offline play
- the game shall support future addition of languages beyond english.
- Fast loading time
- Scalable layout for different resolutions
- Low memory consumption on lower-end mobile devices
- No advertisements or external links
- background music volume control

2.1.2 Prioritization of non-functional requirements

Priority	Non-Functional Requirement
Must	**Simple, easy-to-use UI for young children — Large buttons, clear visuals, minimal text**
Must	**Platform compatibility — Fully functional on both desktop (mouse) and mobile (touch)**
Should	**Smooth animations and visual consistency — Engaging transitions that help maintain focus**
Must	**Safe offline play — No internet needed, safe for unsupervised play**
Must	**Fast loading time — Game starts quickly to prevent loss of attention**
Should	**Scalable layout for different resolutions — Responsive UI adapts to screens of all sizes**
Must	**Low memory consumption on lower-end mobile devices — Lightweight assets and optimized code**
Must	**No advertisements or external links — No risk of accidental taps or distractions**
Could	**Background music volume control**

2.1.3 Architectural Principles

The system design follows key architectural principles to ensure the game is maintainable, scalable, and suitable for young children:

Simplicity
The user interface and gameplay logic are intentionally kept simple to support ease of maintenance and ensure accessibility for children aged 3–7.

Modularity
Core components such as the main menu, game scene, and card objects are implemented as separate scenes (Modular Scene Design). Game logic, UI, input handling, and assets are organized into distinct, reusable modules.

Reusability
Reusable scenes (prefabs) are used for UI elements and cards, allowing consistent behavior and easy duplication or extension in future levels or themes.

Scalability
The architecture allows for easy addition of new cards or difficulty levels without altering core game logic or the engine structure.

Decoupling of UI and Logic
UI and game logic are separated using signal-based communication, improving maintainability and reducing interdependencies between components.

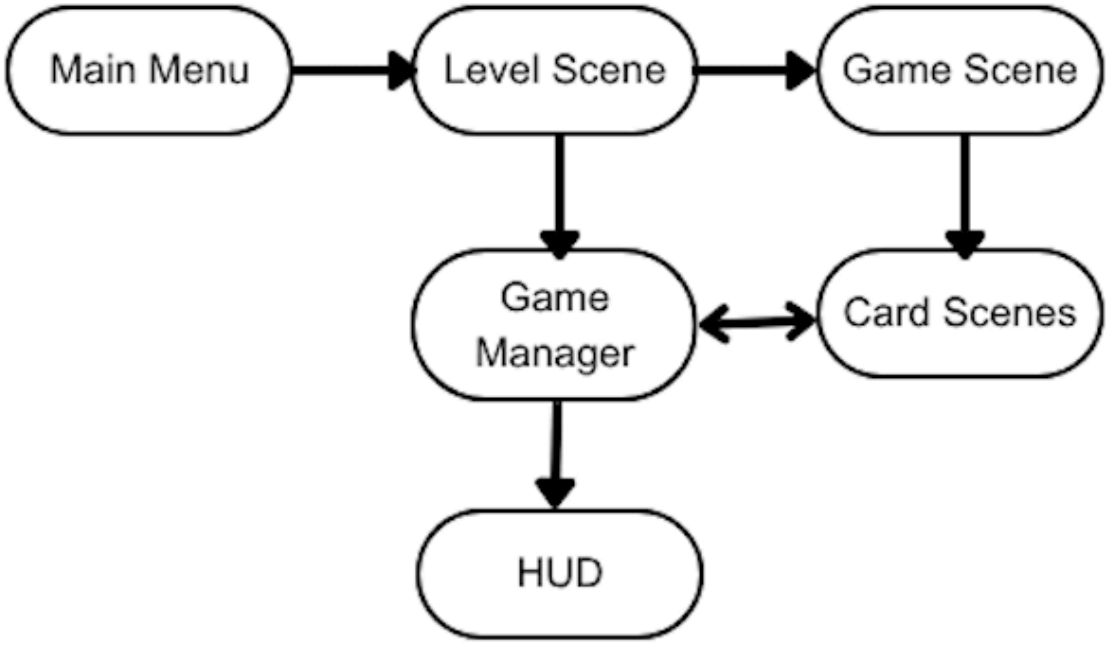
Cross-Platform Compatibility
The game is designed to behave consistently across desktop (mouse input) and mobile (touch input) platforms.

Maintainability
Code and scene structures are kept clean and organized, making it straightforward to expand the game or make future updates (e.g., new UI themes, additional content).

2.1.4 Interfaces

- **User Input Interfaces**: Touch (Android), Mouse (PC)
- **Game UI Interfaces**: Buttons (Start, Difficulty, Quit), Card nodes
- **Platform Interface**: Godot’s export configurations for Android and PC

2.1.5 Big Picture Architecture



- Main Menu → Level Scene: Difficulty is selected and passed to the appropriate level.
- Game Manager acts as the central coordinator, handling logic and state.
- Card Scenes emit signals to GameManager (e.g., when a card is flipped).
- HUD updates visuals (score, game state) based on the manager's data.

2.2 System Design

2.2.1 System Decomposition

- `Main Menu Scene / Script`

  This is the main scene which means that this scene loads up when the game is runned. This scene consists of buttons to traverse between levels and to exit the game. When a level button is clicked, the game gets reset so the values acquired from a scene do not interfere with the current scene.

- **Game Scene / Script**

  Here the card's spawning logic is applied using the grid container. Which means the cards are spawned in an array, and the columns are adjusted in each level by that level's script. Cards are randomized in the setup_cards function. There is a get_total_pairs function which is used to fetch the value to gameManager script.

- **Card Scene / Script:**

  The card's textures, their flipping logic are set here. An animationPlayer is used for the flipping, the textures are swapped right in the middle to show a smooth animation. Each card has a suit and a value, the suit is used to store two of the same texture and the value represents each unique design. The textures are loaded in from the assets via their paths.

- **Level Scenes / Scripts:**

  The scenes for the levels are just subscenes of the game scene, which means that the level scenes do not have their own logic and functions. In their scripts key values that separate each level are declared at the top of the script. Each level has a button that changes the current scene to the main scene.

- **Game Manager Script:**

  Game manager does not have its own scene but in its script most of the game's logic is handled. Values are fetched from scripts all over to be used in the functions here.

  setup_timers function handles the timer for the level time countdown, wait time for the second card pick and the cooldown after two cards are picked to prevent the user from picking more than two cards at the same time.

  register_click function keeps track of the total clicks per level and this value is later used in the point calculation algorithms.

  chooseCard, check_cards, turn_over_cards functions are used for the game logic. match_cards is the function that visually deletes the cards from the screen but in reality the card textures are swapped with a texture that blends in with the background and the interactions with the cards are disabled. Reason for this implementation was because of the grid and array usage with the card spawns, if the cards were actually deleted from the array their places on the array would shift after each successful pairing.

level_is_finished is an indicator method used in other functions, the usage was for quality of life improvement.

it is used instead of: found_pair_count == game_scene.total_pairs

get_click_multiplier and get_time_multiplier are used for calculating the final score. These multipliers are based on the values gathered from the amount of clicks and time used per level, their relation to the average results of our data set created by people that have tried the game.

update_score function is called after every successful match and it has a special case if level_is_finished() and not final_score_applied, which is an indicator for calculation of the final score.

reset_game_state resets all game variables to their initial values for a new game.

- **HUD:**

  In the hud the total clicks and time left on the level are displayed and they are connected to the corresponding methods on the game manager script. Each value is dynamically changed. After the game is finished a pop-up is displayed showing the final score.

2.2.2 Design Alternatives and Decisions

- **Card Pairing Logic**: Total of 3 implementations were considered which were spawning cards individually , using a tile map and spawning them as an array on a grid. The array and grid implementation were chosen because it was easier than handling each card on their own and the visual quality, more fluent animation usage were better compared to the tile map.
- **Difficulty Levels**: Implemented via changing array length displayed on the grid and adjusting the column length of the grid. (e.g., 2x4, 4x4).
- **Animation Choices**: Choose tween-based card flipping for performance and child-friendliness.

2.2.3 Cross-Cutting Concerns (NFR Support)

- **Performance**: Used lightweight textures and preloaded assets for fast loading.
- **Accessibility**: Large buttons, minimal text, vibrant colors to guide attention.
- **Responsiveness**: UI designed using containers and anchors for multi-resolution support.

2.3 Human-Machine Interface (HMI)

2.3.1 Requirements for the H-M Interface

- Large, tappable buttons for small fingers
- Immediate visual feedback
- Easy to return to menu
- Engaging visuals and animations to maintain interest

2.3.2 Design Principles and Style Guide

UI Guidelines :

- Colorful & Friendly: Use soft, bright colors for young children
- Minimal Text: Use icons or very short labels
- Consistency: Similar layouts for each screen
- Visual Feedback: Sounds for success
- Safety: No ads or accidental exit buttons

Typography & Color :

- Font: Mont-Heavy Demo
- Color palette: Soft pastel colors
- Contrast: Ensure text/icons stand out from background

2.3.3 Interaction Modeling